Динамическое программирование. Реализация

Задача о рюкзаке (revisited)

- **Дано:**
  - Набор товаров (n штук), каждый из которых характеризуется весом q_i и стоимостью p_i.
  - Рюкзак, загрузка которого не должна превышать Q_{max}
- **Требуется:**
  - Выбрать набор вещей, обладающих наибольшей стоимостью при условии выполнения требования по максимальной загрузке

Задача о рюкзаке. Уравнение Беллмана

- Описание процесса:
  - Этапы – предметы (n).
  - Выигрыш – стоимость предметов в рюкзаке.
  - Управление – решение о том, брать или не брать предмет ($u_i \in \{0,1\}$).
  - Состояние – остаточная вместимость рюкзака ($S_i \in \{0,..,Q_{max}\}$).
- Уравнение Беллмана:

$$W_i(S_i) = \max_{u_i \in \{u | u \in \{0,1\}, uq_i \leq S_i\}} \{p_i u_i + W_{i+1}(S_i - q_i u_i)\}$$

Цена i-того предмета

Вес i-того предмета

Наивная реализация

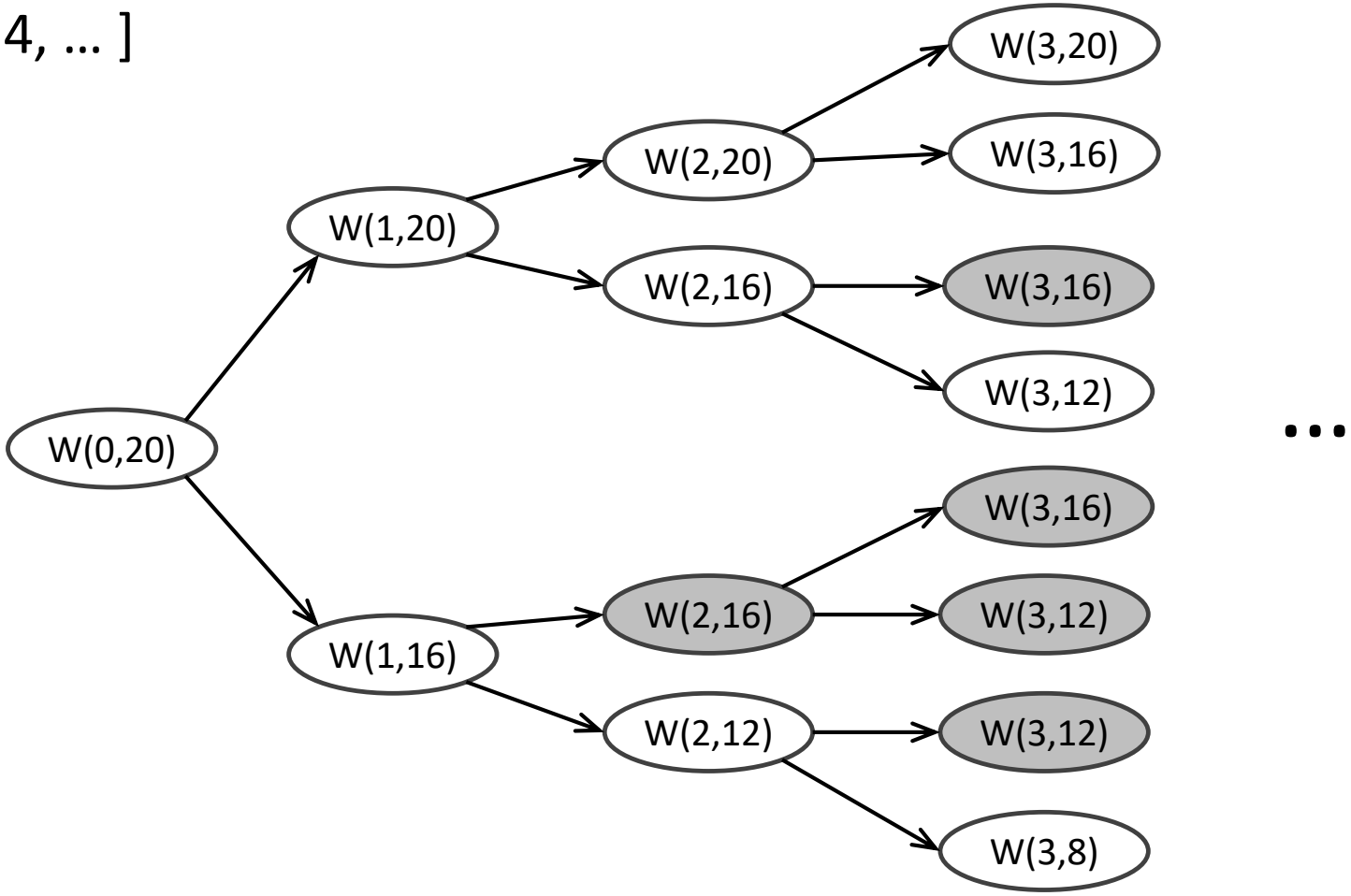
```
class NaiveKnapsackSolver:
    def __init__(self, weights: list[int], prices: list[float], capacity: int):
        self.q = weights
        self.p = prices
        self.c = capacity
        self.n_items = len(self.q)

    def W(self, stage: int, state: int) -> tuple[float, int | None]:
        if stage >= self.n_items:
            return (0, None)

        best_w = None
        best_u = None
        for u in (0, 1):
            if u * self.q[stage] <= state:
                wi = self.p[stage] * u + self.W(stage + 1, state - self.q[stage] * u)[0]
                if best_w is None or wi > best_w:
                    best_w = wi
                    best_u = u
        return (best_w, best_u)

    def solve(self) -> tuple[float, int | None]:
        return self.W(0, self.c)
```

$$W_i(S_i) = \max_{u_i \in \{u | u \in \{0,1\}, uq_i \leq S_i\}} \{p_i u_i + W_{i+1}(S_i - q_i u_i)\}$$

НЕ ИСПОЛЬЗОВАТЬ!

Проблемы наивной реализации

- Пусть:
  - q_i = [4, 4, 4, …]
  - Q_{max} = 20



Проблемы наивной реализации

- Очень неэффективно
  - В первую очередь, из-за многократного пересчета одних и тех же значений
- Нет возможности восстановить оптимальное управление

Наивная реализация. Восстановление управления

```
def W(self, stage: int, state: int) -> tuple[int, list[int]]:

    if stage >= len(self.p):
        return (0, [])

    best_w = None
    best_u = None
    for u in (0, 1):
        if u * self.q[stage] <= state:
            next_w, next_u = self.W(stage + 1, state - u * self.q[stage])
            wi = self.p[stage] * u + next_w
            if best_w is None or wi > best_w:
                best_w = wi
                best_u = [u] + next_u
    return (best_w, best_u)
```

$$W_i(S_i) = \max_{u_i \in \{u | u \in \{0,1\}, uq_i \le S_i\}} \{p_i u_i + W_{i+1}(S_i - q_i u_i)\}$$

НЕ ИСПОЛЬЗОВАТЬ!

Рекурсивная реализация (с мемоизацией)

```
1.    class StandardKnapsackSolver:
2.        def __init__(self, weights: list[int], prices: list[float], capacity: int):
3.            self.q = weights
4.            self.p = prices
5.            self.c = capacity
6.            self.n_items = len(self.q)
7.            self.W_cache = [{} for _ in range(self.n_items)]

8.        def W(self, stage: int, state: int) -> tuple[float, int | None]:
9.            if stage >= self.n_items:
10.               return (0, None)

11.           if state in self.W_cache[stage]:
12.               return self.W_cache[stage][state]

13.           best_w = None
14.           best_u = None
15.           for u in (0, 1):
16.               if u * self.q[stage] <= state:
17.                   wi = self.p[stage] * u + self.W(stage + 1, state - self.q[stage] * u)[0]
18.                   if best_w is None or wi > best_w:
19.                       best_w = wi
20.                       best_u = u
21.           return (best_w, best_u)
```

$$W_i(S_i) = \max_{u_i \in \{u | u \in \{0,1\}, uq_i \le S_i\}} \{p_i u_i + W_{i+1}(S_i - q_i u_i)\}$$

Рекурсивная реализация (с мемоизацией)

…продолжение

```
def restore_optimal(self) -> list[int]:
    control = []
    state = self.c
    for stage in range(self.n_items):
        u = self.W(stage, state)[1]
        control.append(u)
        state = state - u * self.q[stage]
    return control

def solve(self) -> tuple[float, list[int]]:
    return (self.W(0, self.c)[0],
            self.restore_optimal())
```

Рекурсивная реализация (с мемоизацией). Декораторы

```
1.  from functools import cache

2.  @cache
3.  def W(self, stage, state):
4.      …
```

Python < 3.9

```
1.  from functools import lru_cache

2.  @lru_cache(maxsize=None)
3.  def W(self, stage, state):
4.      …
```

При применении к методам класса могут возникать утечки!

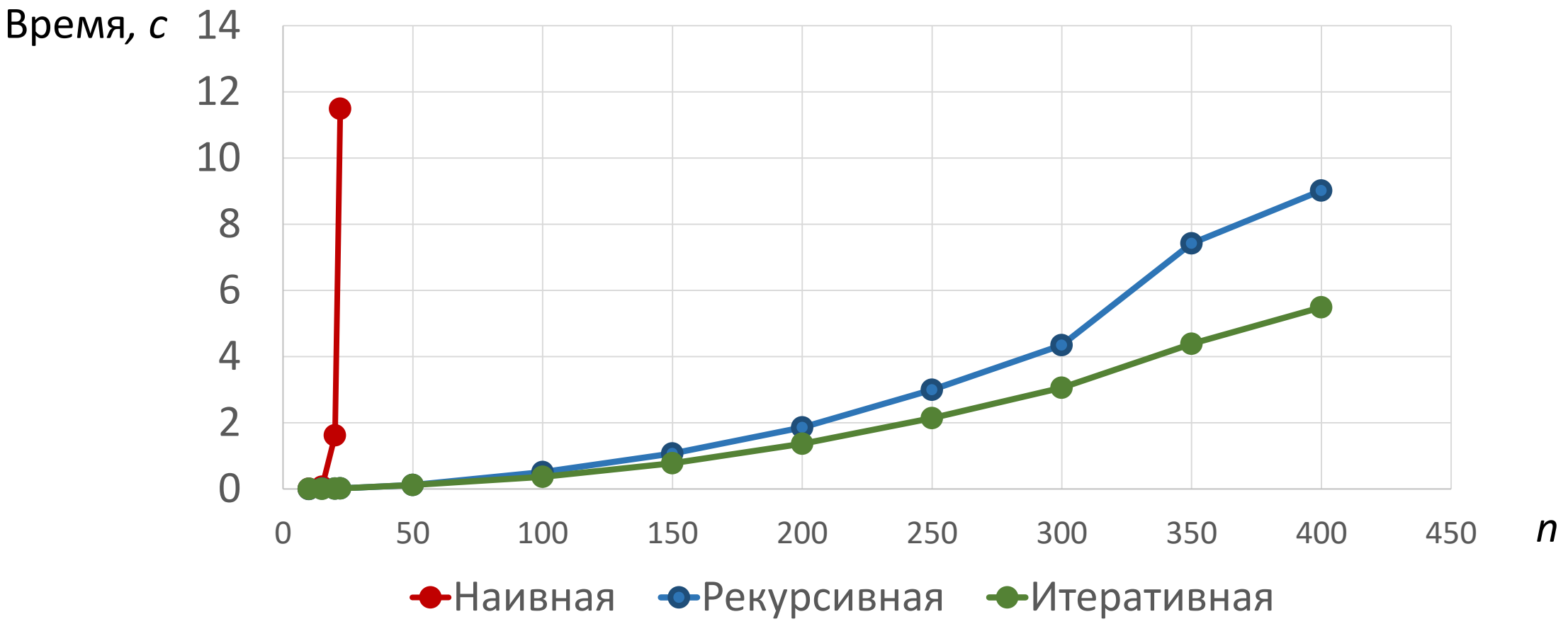
Итеративная реализация

$$W_i(S_i) = \max_{u_i \in \{u | u \in \{0,1\}, uq_i \le S_i\}} \{p_i u_i + W_{i+1}(S_i - q_i u_i)\}$$

```
def calculate(self):
    self.W_cache = [{} for _ in range(self.n_items)]

    # Фиктивный шаг
    self.W_cache.append({})
    for state in range(0, self.c + 1):
        self.W_cache[self.n_items][state] = (0, None)

    for stage in reversed(range(self.n_items)):
        for state in range(0, self.c + 1):
            best_w = None
            best_u = None
            for u in (0, 1):
                next_state = state - self.q[stage] * u
                if next_state >= 0:
                    wi = self.p[stage] * u + self.W_cache[stage + 1][next_state][0]
                    if best_w is None or wi > best_w:
                        best_w = wi
                        best_u = u
            self.W_cache[stage][state] = (best_w, best_u)
```

Сравнение времени выполнения

Вес и стоимость предмета ~U(5; 50), грузоподъемность рюкзака ~1/2 суммарного веса предметов.



Стилистические улучшения

- Разделение **описания задачи**:
  - Количество этапов
  - Множество состояний на каждом из этапов
  - Множество допустимых управлений в каждом из состояний
  - «Эффекты» управления
- …и обобщенного кода, реализующего схему ДП в соответствующих терминах:

```
def W(…):
        …
        for (u, val, phi) in problem.valid_controls(stage, s):
            wi = val + self.W(stage + 1, phi)
        …
```

Тонкости при работе с непрерывным состоянием

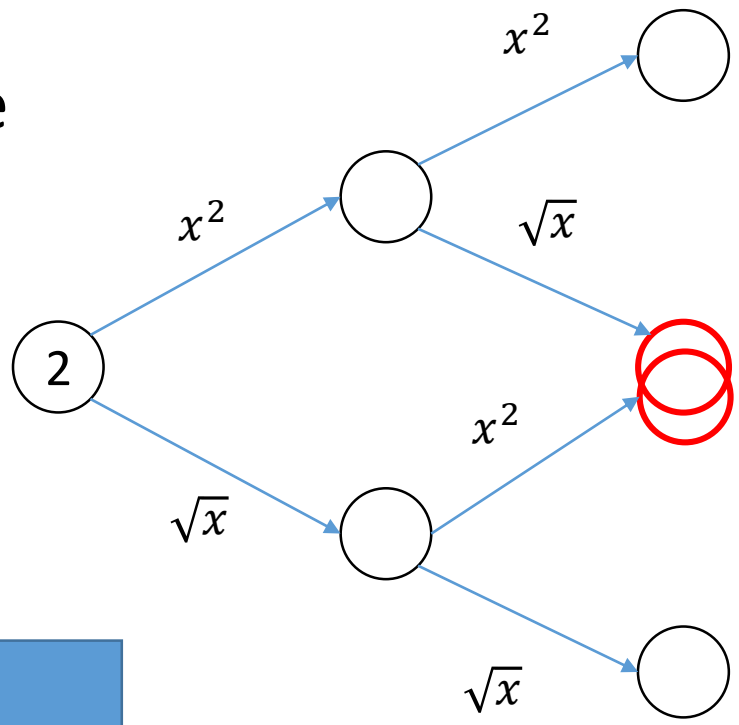
- Простейший вариант – дискретизация
- Использование вещественного ключа в словаре

```
phi = … # некоторая функция от state и u
wi = … + self.W_cache[stage + 1][phi][0]
```

```
2 == math.sqrt(2) ** 2          -> False!
0.1 + 0.1 + 0.1 - 0.3 == 0      -> False!
```

```
a = {}
a[2] = 'Two'
a[math.sqrt(2) ** 2] = 'Two'
```

```
{2: 'Two',
2.0000000000000004: 'Two'}
```



Контролируйте размер словаря!

Тонкости при работе с непрерывным состоянием

- Использование вещественного ключа в словаре. Возможное (но не единственное!) решение – округление:

```
phi = … # некоторая функция от state и u
wi = … + self.W_cache[stage + 1][round(phi, 2)][0]
```

```
round(2, 2) == round(math.sqrt(2) ** 2, 2)          -> True
round(0.1 + 0.1 + 0.1 - 0.3, 2) == round(0, 2)      -> True
```

```
a = {}
a[round(2, 2)] = 'Two'
a[round(math.sqrt(2) ** 2, 2)] = 'Two‘
```

```
{2: 'Two'}
```

Тонкости при работе с непрерывным состоянием

```
class FloatAwareDict:
    def __init__(self, precision):
        self.precision = precision
        self.dict = {}
    def __setitem__(self, k, v):
        self.dict[round(k, self.precision)] = v
    def __getitem__(self, k):
        return self.dict[round(k, self.precision)]
    def __iter__(self):
        return self.dict.__iter__()

fad = FloatAwareDict(2)
fad[0.1 + 0.1 + 0.1 - 0.3] = 'Zero'
fad[0] -> 'Zero'
fad[0.0000001] -> 'Zero'
```

Резюме

- Существует два базовых способа программной реализации применения метода динамического программирования
  - Итеративный
  - Рекурсивный
- Итеративный
  - (-) Возможно, требует больше памяти
  - (-) Чуть сложнее в реализации
  - (+) Как правило, работает несколько быстрее
- Рекурсивный
  - (+) Может сэкономить память
  - (+) Чуть проще в реализации
  - (-) Как правило, работает чуть медленнее
  - (-) Можно «упереться» в ограничения стека

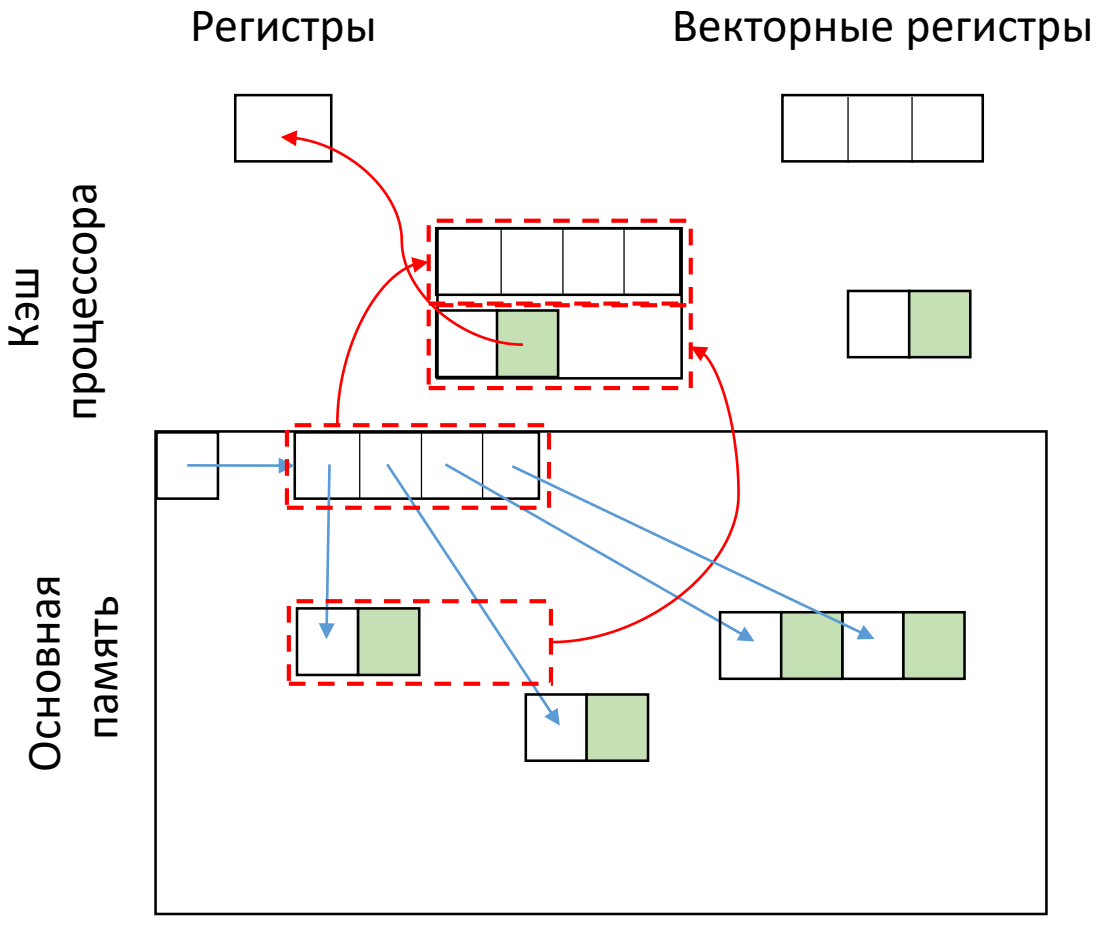
Дополнение

Векторизованная реализация

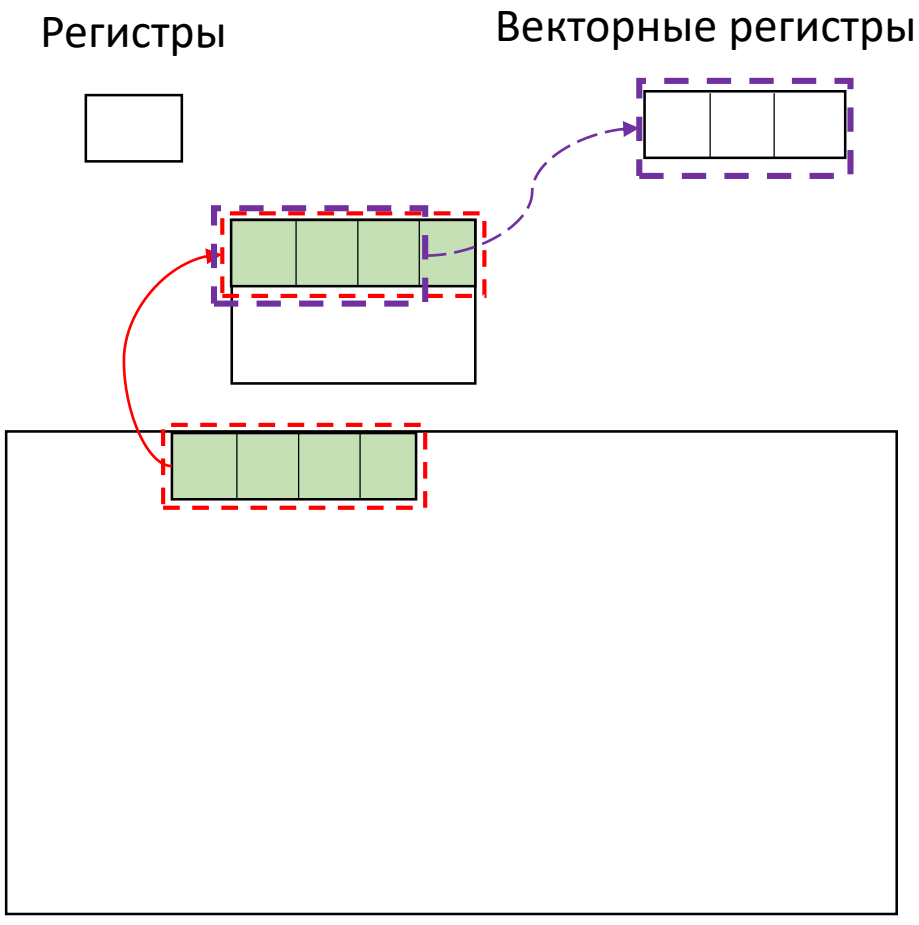
- Реализации выше – рекомендованы, основное их достоинство – гибкость:
  - Любое состояние может быть сохранено как ключ в кортеже
- Однако вычисления на Python не очень эффективны. Причины:
  - Интерпретируемый язык
  - Размещение объектов в памяти
  - Отсутствие поддержки векторных инструкций
- Решение (частичное):
  - Использование numpy

Рекомендуемая статья: https://pythonspeed.com/articles/performance-memory-locality/

Векторизованная реализация «под капотом»



Python



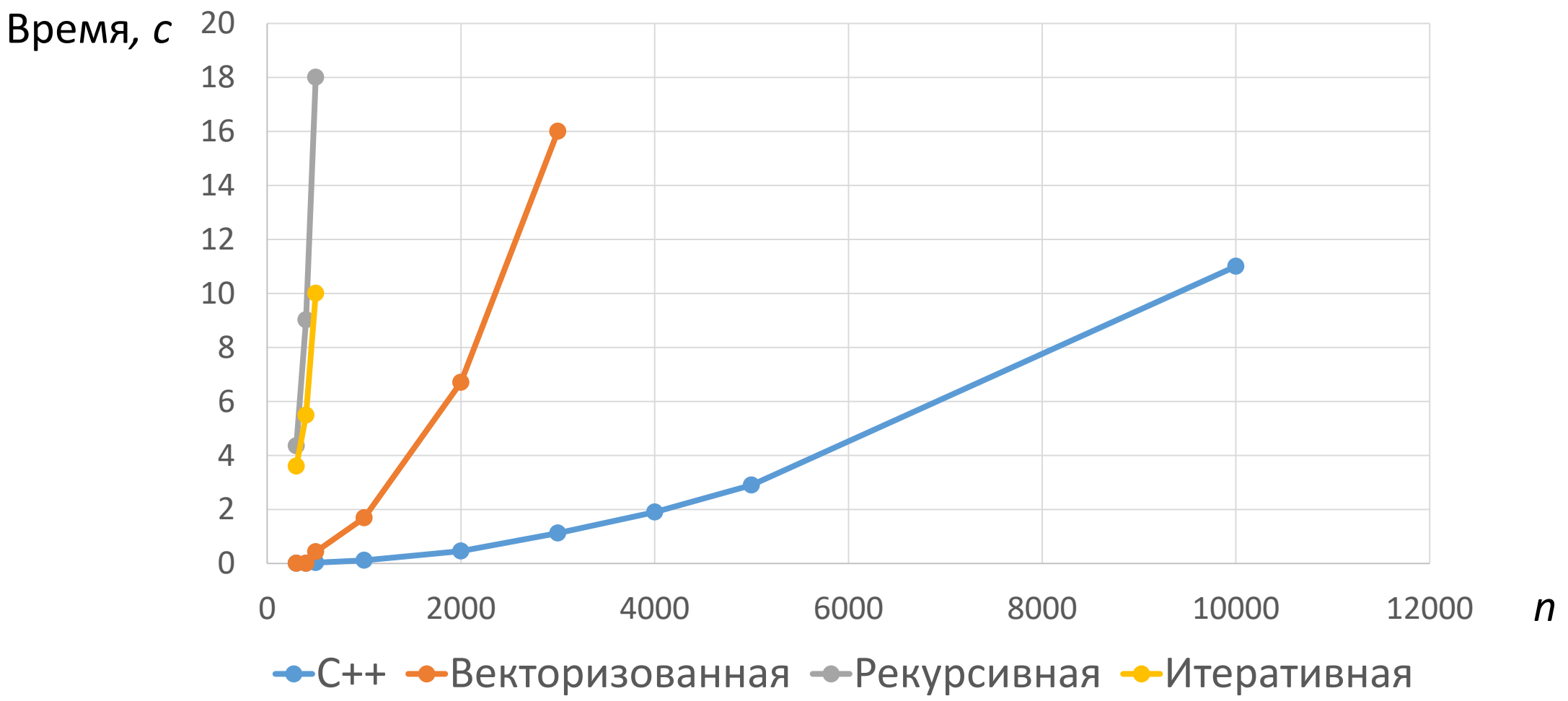
Компилируемые языки, numpy (!)

Векторизованная реализация

```
class VectorizedIterativeNPKnapsackSolver:
    def __init__(self, weights, prices, capacity):
        self.q = weights
        self.p = prices
        self.c = capacity
        self.W_cache = [np.zeros(self.c+1) for _ in self.q]
        self.U_cache = [np.zeros(self.c+1, dtype=np.int32)
                        for _ in self.q]
```

```
    def calculate(self):
        # Фиктивный шаг
        self.W_cache.append(np.zeros(self.c + 1))
        self.U_cache.append(np.zeros(self.c + 1, dtype=np.int32))

        NINF = -1000
        # Инициализация векторов управления и состояния
        x = np.array([0, 1])
        s = np.arange(self.c + 1)
        # Преобразование их в вид, подходящий для вычислений
        xx = np.broadcast_to(x.reshape(1, -1), (self.c + 1, len(x)))
        ss = np.broadcast_to(s.reshape(-1, 1), (self.c + 1, len(x)))

        for stage in reversed(range(len(self.q))):
            # Вычисление новых состояний
            new_state = ss - self.q[stage] * xx
            valid_state = new_state >= 0
            new_state[~valid_state] = 0

            tmp_w = self.p[stage] * xx + self.W_cache[stage + 1][new_state]
            tmp_w[~valid_state] = NINF

            u = np.argmax(tmp_w, axis=1, keepdims=True)
            W = np.take_along_axis(tmp_w,
                                   u,
                                   axis=-1)

            self.W_cache[stage] = W.ravel()
            self.U_cache[stage] = u.ravel()
```

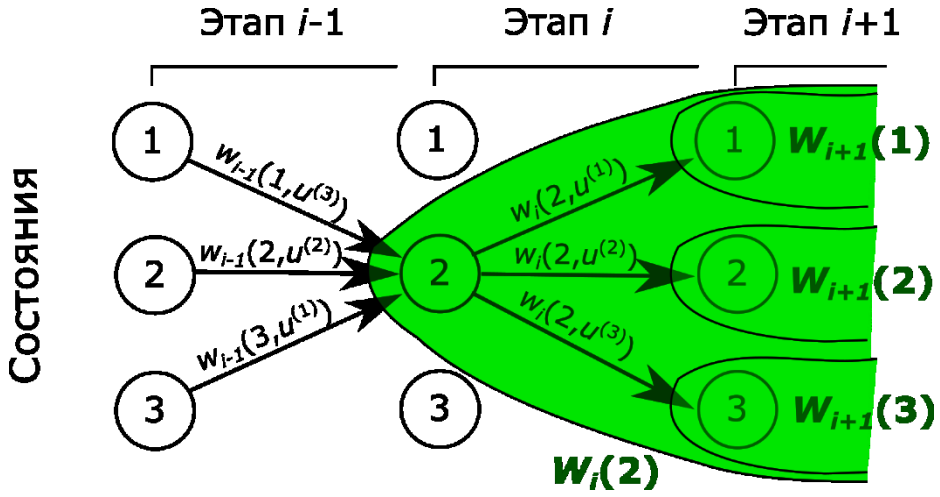
Еще о времени выполнения



Уравнения прямой и обратной прогонки

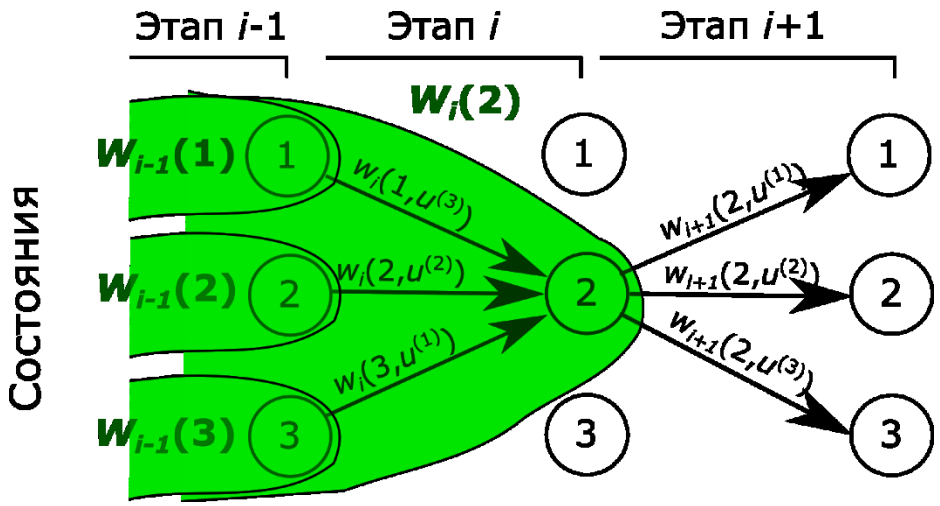
Уравнение обратной прогонки:
Условно оптимальный выигрыш $W_{i[:n]}(S_i)$ - максимальный суммарный выигрыш, который может быть получен на всех шагах, начиная с *i*-того, если к *i*-тому шагу система находится в состоянии S_i:

$$W_{i[:n]}(S_i) = \max_{u_i \in U_i(S_i)} \{w_i(S_i, u_i) + W_{i+1[:n]}(\phi(S_i, u_i))\}$$



Уравнение прямой прогонки:
Условно оптимальный выигрыш $W_{[0:]i}(S_i)$ - максимальный суммарный выигрыш, который может быть получен на всех шагах до *i*-того (включительно), если к *i*-тому шагу система оказывается в состоянии S_i:

$$W_{[0:]i}(S_i) = \max_{\substack{S_{i-1} \in \mathcal{S}_{i-1} \\ S_i = \phi(S_{i-1}, u_i)}} \{W_{[0:]i-1}(S_{i-1}) + w_i(S_{i-1}, u_i)\}$$



Итеративная реализация прямой прогонки

$$W_{[0:]i}(S_i) = \max_{\substack{S_{i-1} \in \mathcal{S}_{i-1} \\ S_i = \phi(S_{i-1}, u_i)}} \left\{ W_{[0:]i-1}(S_{i-1}) + w_i(S_{i-1}, u_i) \right\}$$

```
def calculate(self):
    self.W_cache = [{} for _ in self.q]

    self.W_cache.append({})
    self.W_cache[0][self.c] = (0, None)

    for stage in range(1, len(self.q) + 1):
        for state_prev, (W_prev, _) in self.W_cache[stage-1].items():
            for u in (0, 1):
                state = state_prev - self.q[stage-1] * u
                if state >= 0:
                    wi = W_prev + self.p[stage-1] * u
                    if state not in self.W_cache[stage] or \
                        self.W_cache[stage][state][0] < wi:
                        self.W_cache[stage][state] = (wi, u)
```

Прямая или обратная прогонка?

- Оба уравнения дают одинаковый результат и обладают одинаковой асимптотической сложностью
- Обратная прогонка
  - Выглядит «естественнее» в задачах планирования – она напрямую соответствует идее «выбора решения с учетом последствий»
  - Допускает достаточно простое расширение до принятия решения в вероятностных процессах
  - Достаточно легко адаптируется для процессов управления с бесконечным горизонтом планирования (с помощью коэффициента дисконтирования)
- Прямая прогонка
  - Может быть (или казаться) более понятной, поскольку уравнение не включает размышление о «зыбких» «будущих выигрышах»
  - Более естественна для некоторых задач (например, алгоритм Форда-Беллмана для поиска кратчайших путей в графе)